

RAG-Based LLM Code Review Agent

Data Science and AI Lab Project

Milestone 2 Report Group 1

Name	Email
Jeevika S	21f3001259@ds.study.iitm.ac.in
Budhil Nigam	23f1001585@ds.study.iitm.ac.in
Kannan S	21f3000990@ds.study.iitm.ac.in
Omkar	22f2001265@ds.study.iitm.ac.in
Karunesh	221001606@ds.study.iitm.ac.in



Contents

1. Objective.....	3
2. Dataset Overview.....	4
2.1. Dataset Sources.....	4
2.2 Evaluation Dataset.....	5
2.3 Retrieval Corpus.....	6
2.4 Static Analysis Input Dataset.....	7
3. Dataset Construction and Description.....	8
3.1 GitHub API Data Collection.....	8
3.2 Raw Data Sources.....	9
3.3 Synthetic Data Generation Strategy.....	10
3.4 Dataset Statistics.....	11
4. Dataset Quality Assessment.....	12
5. Dataset Splitting Strategy.....	13
6. Retrieval Corpus Preparation.....	14
6.1 Document Collection.....	14
6.2 Chunking Strategy.....	15
6.3 Embedding Generation.....	16
6.4 Vector Database.....	16
Visualisations.....	16
7. Data Preprocessing Pipeline.....	18
8. Proposed Methodology Overview.....	19
8.1 System Architecture.....	19
8.2 Scope Restriction.....	20
8.3 Dataset.....	21
8.4 Evaluation Metrics.....	21
9. Expected Outcomes.....	23

1. Objective

The objective of Milestone 2 is to identify, construct, and prepare the datasets required for building a Retrieval-Augmented Code Review Assistant for Python pull requests.

The proposed system analyzes pull request (PR) diffs and automatically detects five categories of coding violations:

- indentation inconsistencies
- naming convention violations
- unused imports
- mutable default arguments
- documentation and formatting issues

The system compares three different approaches:

1. Static analysis tools
2. Large Language Model (LLM) without retrieval
3. Retrieval Augmented Generation (RAG) based code review

To support this evaluation, datasets were collected and constructed from GitHub repositories, coding guidelines, and linter rule documentation.

The data preparation process focuses on:

- identifying reliable data sources
- constructing structured datasets from raw pull request diffs
- ensuring dataset quality and consistency
- preparing retrieval knowledge for the RAG architecture
- creating evaluation datasets with ground-truth annotations

2. Dataset Overview

The project generates three primary datasets, each serving a specific purpose in the system pipeline.

Dataset	Purpose
Evaluation Dataset	Used for evaluating code review models
Retrieval Corpus	Knowledge base used by the RAG system
Static Analysis Input	Used to run baseline linters

Dataset	Entries	Organic	Synthetic
Evaluation Dataset	158	128	30
Retrieval Corpus	220	220	0
Static Analysis Input	125	95	30

2.1. Dataset Sources

The pull request data used for constructing the evaluation dataset was collected from several well-established open-source Python repositories. These repositories were selected because they follow strong coding standards, maintain active development communities, and contain extensive pull request review activity.

The repositories used in this study include:

Repository	Role
django/django	Training

pandas-dev/pandas	Training
scikit-learn/scikit-learn	Training
pallets/flask	Evaluation
fastapi/fastapi	Evaluation

These repositories provide a diverse set of Python codebases across web development, machine learning, and data processing domains.

2.2 Evaluation Dataset

Source: Public Python repositories on GitHub.

Description:

The evaluation dataset contains pull request diffs extracted from open-source repositories. These diffs represent real code changes submitted by developers during collaborative development.

Metric	Value
Total entries	158
Organic entries	128
Synthetic entries	30
Total violations	268

Violation distribution:

Category	Count
indentation	104
documentation_formatting	75
naming_convention	53
mutable_default	21

unused_import	15
---------------	----

Each pull request diff is segmented into diff chunks, which are annotated with ground truth review comments describing coding violations.

```
{
  "pr_id": "PR_101",
  "repo": "psf/requests",
  "file_path": "utils/helpers.py",
  "diff_chunks": [
    {
      "chunk_id": "c1",
      "start_line": 10,
      "end_line": 25,
      "diff_lines": [
        "+ import numpy as np",
        "+ import math",
        "+",
        "+ def area(radius):",
        "+     return math.pi * radius * radius"
      ]
    }
  ],
  "ground_truth_reviews": [
    {
      "line_number": 10,
      "violation_category": "unused_import",
      "review_comment": "numpy is imported but never used."
    }
  ]
}
```

2.3 Retrieval Corpus

The retrieval corpus forms the knowledge base used by the RAG model. Each document is converted into knowledge chunks that contain rule explanations related to code quality.

Sources include 220 knowledge chunks of:

- Python coding guidelines
- project style guides
- linter rule explanations
- historical review comment

Source	Chunks
Review comments	84

PEP-8 guidelines	81
Linter rules	36
PEP-257 documentation	17
Project guidelines	2

Chunk size statistics:

- min: 9 words
- max: 421 words
- mean: 122 words

Chunks were split to remain within **400 tokens**, ensuring efficient retrieval.

Example:

```
{
  "chunk_id": "guideline_32",
  "text": "Avoid importing modules that are not used in the file. Unused imports make code harder to read.",
  "category": "unused_import",
  "source_type": "project_guideline"
}
```

These chunks are embedded and stored in a vector database for semantic retrieval.

2.4 Static Analysis Input Dataset

This dataset is used to run baseline static analysis tools such as:

- Flake8
- Pylint

Example structure:

```
{
  "file_path": "utils/helpers.py",
  "diff_code": "import numpy as np\nimport math\n..."
}
```

These tools generate rule-based violation reports that serve as a baseline for comparison with LLM-based systems.

Statistics:

Metric	Value
Files	125
Organic files	95
Synthetic files	30
Lines of code	39,015

3. Dataset Construction and Description

The datasets used in this study are constructed through a **multi-stage data collection and transformation pipeline**. The pipeline gathers raw pull request data, code changes, and review comments from open-source repositories and converts them into structured datasets that can be used by the automated code review system.

3.1 GitHub API Data Collection

Data for the evaluation dataset was collected using the GitHub REST API (v3). The collection process systematically retrieved pull request review comments and file change diffs across selected repositories.

The following endpoints were used during data collection:

PR Review Comments

[GET /repos/{owner}/{repo}/pulls/comments]

This endpoint was used to retrieve the most recent review comments from pull requests.

PR File Changes

[GET /repos/{owner}/{repo}/pulls/{pr}/files]

This endpoint was used to extract unified diff patches representing code modifications in each pull request.

To ensure reliable and stable data collection, several strategies were implemented:

- Pagination was used to retrieve large datasets across multiple API responses.
- A 1-second delay between API requests was introduced to respect GitHub rate limits.
- Automatic retries were implemented when HTTP 403 rate-limit responses were encountered.
- All API responses were cached locally in a cache/ directory to ensure experiment reproducibility.

In total, the data collection process retrieved:

- 25,000 pull request review comments
- 98 pull request file change sets
- Data spanning five large Python repositories

3.2 Raw Data Sources

The raw dataset is constructed from several complementary sources that provide information about code changes and coding standards.

Raw sources include:

- GitHub pull request diffs
- repository style guidelines
- Python style rules
- linter rule documentation

The collected data is processed and transformed into structured JSON datasets used by the system.

The data collection pipeline used the following GitHub API endpoints:

- /repos/{owner}/{repo}/pulls/comments
- /repos/{owner}/{repo}/pulls/{number}/files
- /repos/{owner}/{repo}/contents/{path}

Important features in the dataset include:

Feature	Description
PR ID	Unique identifier assigned to each pull request in the dataset

Repository	Name of the GitHub repository from which the pull request was extracted
File Path	The path of the modified file within the repository
Diff Chunks	Segmented portions of the pull request diff representing code changes
Line Numbers	Line numbers indicating the location of the modification in the file
Violation Category	The category of coding issue detected (e.g., unused import, indentation error)
Review Comment	Ground truth review comment describing the detected violation

3.3 Synthetic Data Generation Strategy

During the data collection phase, pull request review comments were scraped from several open-source Python repositories using the GitHub API. In total, more than **25,000 review comments** were collected across the selected repositories. However, only a very small fraction of these comments corresponded to **actual coding guideline violations** relevant to the scope of this project.

Most pull request comments typically relate to general development workflow or project coordination, such as suggestions about architecture, requests for additional tests, clarification questions, or general feedback (e.g., “Looks good”, “Please add tests”, “Can you rebase?”). These comments do not correspond to identifiable coding rule violations and therefore, cannot be used as labeled examples for a code review violation detection system.

After filtering and preprocessing, only a limited number of comments could be confidently mapped to the five predefined violation categories used in this project:

- indentation inconsistencies
- naming convention violations
- unused imports
- mutable default arguments
- documentation or formatting issues

Because of this imbalance, relying solely on the filtered GitHub data would result in **severe category imbalance**, where some violation types appear frequently while

others appear only rarely. Such an imbalance negatively affects both retrieval quality and evaluation reliability.

To address this limitation, **synthetic examples were generated selectively** to supplement the real dataset. Synthetic data was created only for categories that were significantly underrepresented in the collected GitHub review comments. The generation process produced realistic examples of:

- code diffs representing a specific violation pattern
- corresponding review comments describing the violation

These synthetic examples were designed to follow Python coding standards (PEP8 and PEP257) and typical phrasing patterns found in real code review discussions.

To minimize bias introduced by artificial examples, the following constraints were applied:

1. **Real GitHub data remains the dominant portion of the dataset.**
The majority of examples originate from actual pull request reviews.
2. **Synthetic data is used primarily to balance rare categories.**
It is introduced only where the collected data lacks sufficient representation.
3. **The evaluation dataset prioritizes real-world examples.**
Synthetic examples are included only in limited numbers to ensure category coverage.

This strategy allows the dataset to maintain a realistic representation of GitHub code review behavior while ensuring that each violation category is sufficiently represented for meaningful experimentation. Future iterations of the dataset can progressively replace synthetic examples with additional real-world data as more labeled examples are collected.

3.4 Dataset Statistics

The raw dataset collected during the pipeline consists of:

Dataset Component	Count
Review Comments	25000
Pull Request File Changes	98
Repositories	5

Linter Rule Explanations	36
PEP Guideline Sections	Multiple

These datasets are then transformed into the structured evaluation dataset and retrieval corpus used by the RAG system.

4. Dataset Quality Assessment

Dataset quality was evaluated to ensure that the data is suitable for automated code review analysis.

Comment Classification System

To identify violation comments from the raw review dataset, a hybrid classification approach was used.

1. LLM Classification: Review comments were classified using GPT-4 based models.

To minimize API cost and token usage:

- Comments were batched (80 per request)
 - Multiple models were load-balanced
 - Classification prompts emphasized precision over recall.
 - Comments that did not clearly match a violation were labeled "none".
2. Regex Fallback Classifier: A regex-based classifier was implemented as a fallback when LLM responses were unavailable or malformed.
 - Over 50 keyword patterns were used across the five categories.
 3. Cross Validation
 - A second model independently re-classified evaluation comments. Disagreements were handled conservatively to ensure label reliability.

Missing Values

Potential missing values include:

- missing line numbers
- incomplete diff chunks

- absent annotations

These entries were filtered or manually verified to maintain dataset reliability.

Duplicate Entries

Duplicate code diffs can occur when similar code changes appear in multiple pull requests.

Duplicates were removed to prevent evaluation bias.

Noise and Inconsistencies

Noise may arise from:

- formatting artifacts in GitHub patch files
- unrelated code changes
- comments or metadata inside diffs

Preprocessing steps normalize indentation and remove irrelevant lines.

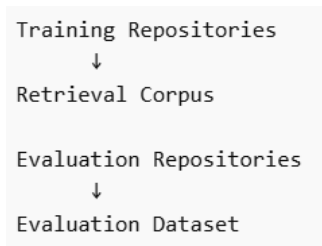
5. Dataset Splitting Strategy

Although the project does not train model weights directly, dataset splitting is necessary to avoid data leakage in the retrieval system.

The split strategy is repository-based.

Training repositories are used to construct the retrieval corpus.

Evaluation repositories are used exclusively for building the evaluation dataset.



This separation ensures that the system cannot retrieve ground truth answers during evaluation.

6. Retrieval Corpus Preparation

The retrieval corpus is designed to provide **contextual knowledge for the Retrieval-Augmented Generation (RAG) model**. This corpus contains coding guidelines, rule explanations, and documentation that help the model understand coding standards when generating review comments.

Documents are processed through several stages including **collection, chunking, embedding generation, and indexing**.

6.1 Document Collection

Style Guide Sources

The retrieval corpus was constructed from multiple coding guideline sources to ensure **comprehensive coverage of Python coding standards and best practices**.

The corpus integrates both **general Python style documentation and project-specific coding rules**.

Sources include the following categories:

PEP Documentation

Official Python Enhancement Proposals (PEPs) provide the primary reference for Python coding conventions.

- PEP 8 – Python coding style guidelines covering indentation, naming conventions, import organization, whitespace usage, and formatting rules.
- PEP 257 – Docstring conventions describing how documentation strings should be structured and formatted.

Project Contribution Guidelines

Repository-specific style documentation was also included to capture **real-world coding practices used by large open-source projects**.

Documentation was extracted from files such as:

- CONTRIBUTING.md

- STYLE_GUIDE.md
- Developer documentation

These documents were collected from major Python repositories including:

- django/django
- pandas-dev/pandas
- scikit-learn/scikit-learn

These sources provide additional project-specific coding conventions beyond general Python guidelines.

Lint Rule Documentation

Static analysis rule descriptions were included to represent **automated coding standard checks used in practice**.

Rule explanations were collected from:

- Pylint rule descriptions
- Flake8 rule explanations

A total of **36 rule explanations** covering the five targeted violation categories were included in the corpus.

6.2 Chunking Strategy

To improve retrieval efficiency, documents are divided into smaller segments before embedding.

Each document is split into 200–400 token chunks. Smaller chunks allow the embedding model to capture specific coding rules and concepts more precisely, improving semantic retrieval performance.

Example chunk:

*Mutable default arguments should be avoided because
The same object persists across function calls.*

This chunking strategy ensures that each vector represents a focused coding guideline rather than an entire document.

6.3 Embedding Generation

In future versions of the system, textual guideline chunks may be converted into vector embeddings using models such as **bge-large-en-v1.5**.

Embeddings represent the semantic meaning of coding rules and allow similarity comparison between code diffs and guideline documents.

This enables the system to identify relevant coding standards that relate to a specific code change.

6.4 Vector Database

The generated embeddings can be stored in a **FAISS vector database** to enable efficient semantic search.

During inference, code diffs could be embedded and used to retrieve the most relevant guideline chunks from the index.

These retrieved documents would then be provided as context to the RAG model for generating grounded review comments.

Visualisations

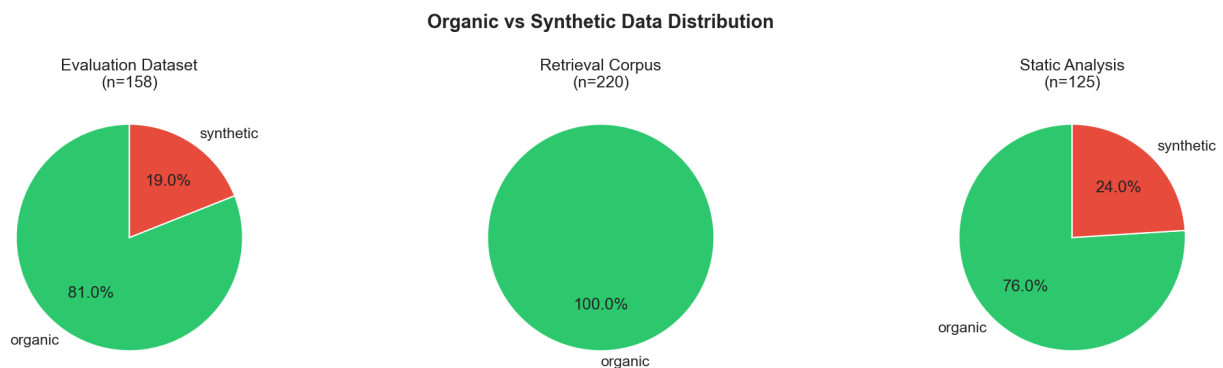


Fig 1: Organic vs Synthetic Data distribution in Evaluation, Retrieval and Static Analysis datasets

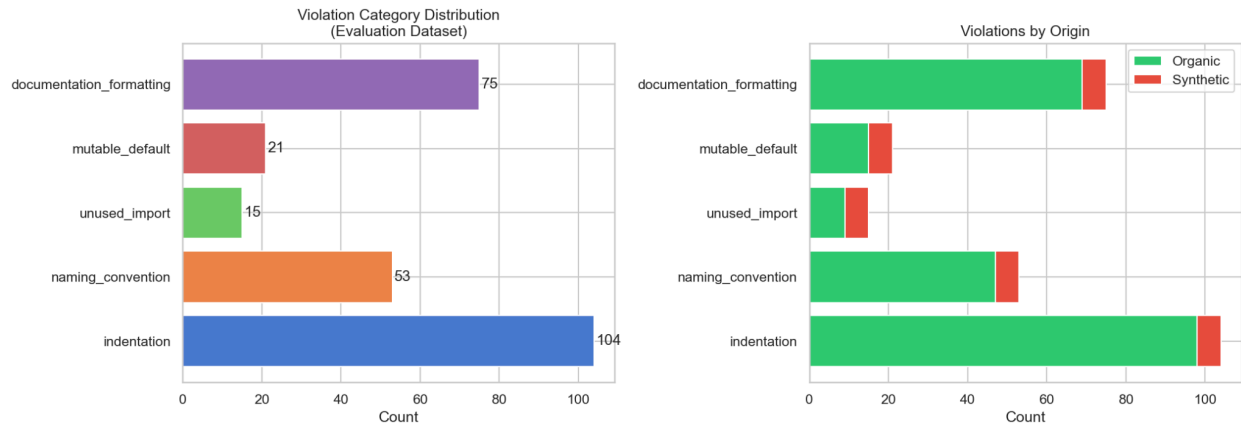


Fig 2: Violation Categories distribution

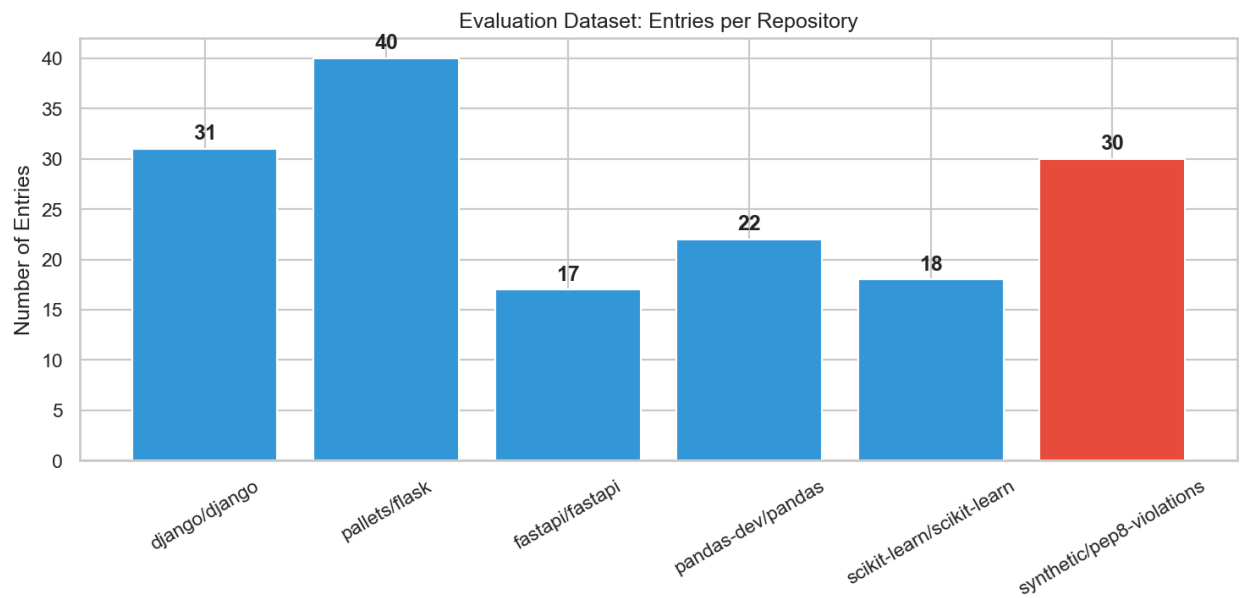


Fig 3: Code Repository wise entries distribution

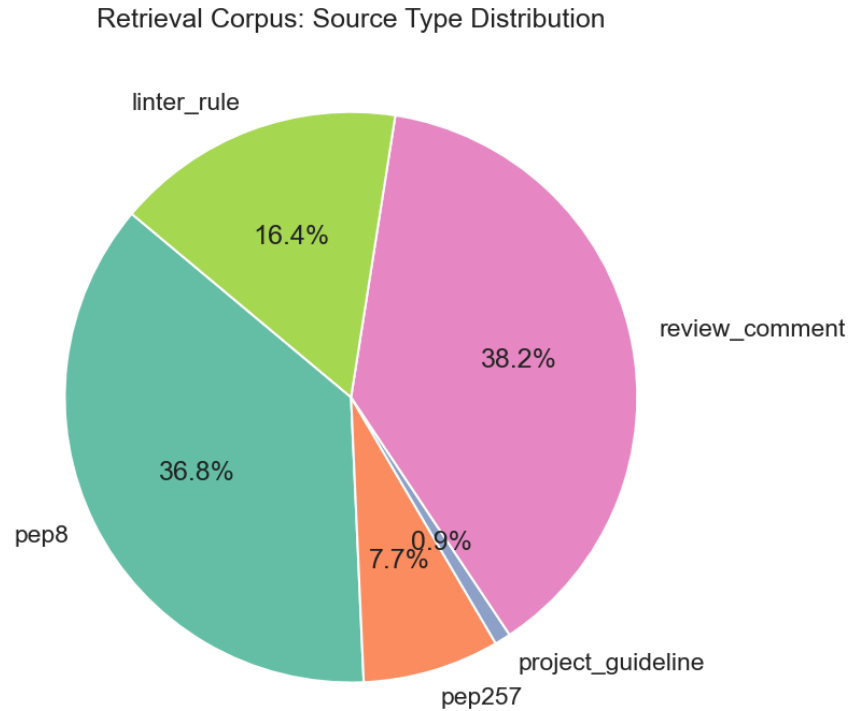


Fig 4: Retrieval Corpus Dataset (Source Type) distribution

7. Data Preprocessing Pipeline

The retrieval corpus is designed to provide contextual knowledge for the Retrieval-Augmented Generation (**RAG**) model. This corpus contains coding guidelines, rule explanations, and documentation that help the model understand coding standards when generating review comments.

Documents are processed through several stages including collection, chunking, embedding generation, and indexing.

1. Fetch

Retrieve raw pull request review comments and file diffs using the GitHub API.

2. Extract

Parse JSON responses and extract structured fields including:

- file path

- line number
- diff hunk
- review comment body

3. Classify

Each review comment is automatically classified into one of the five violation categories using multiple large language models:

- GPT-4.1
- GPT-4o
- GPT-4.1-mini

In cases where API limits are reached, a fallback pattern-matching classifier is applied.

4. Filter

Only comments corresponding to actual guideline violations are retained. General discussion comments are discarded.

5. Construct

Violations are grouped by:

(repository, pull request, file)

and converted into structured evaluation dataset entries with associated diff chunks.

6. Validate

Quality checks are performed including:

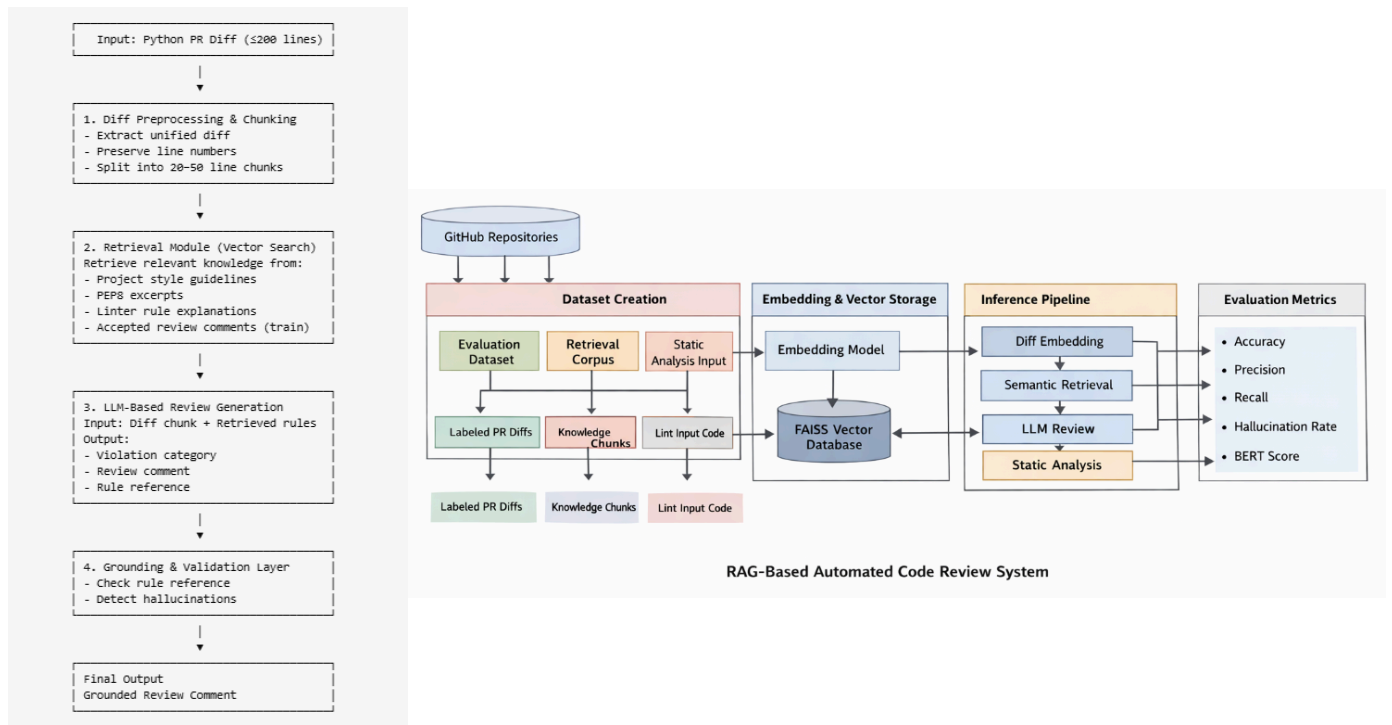
- duplicate removal
- consistency checks
- annotation validation

8. Proposed Methodology Overview

8.1 System Architecture

The proposed system is designed as a Retrieval-Augmented Generation (RAG) framework for automated review of small Python pull request (PR) diffs. The architecture integrates structured retrieval of project-specific coding guidelines

with Large Language Model (LLM)-based review generation to improve correctness, grounding, and contextual alignment.



8.2 Scope Restriction

The system is strictly limited to detecting the following five guideline violation categories:

1. Indentation inconsistencies
2. Naming convention violations
3. Unused imports
4. Mutable default arguments
5. Documentation or formatting deviations

The system explicitly excludes:

- Functional correctness issues
- Security vulnerabilities
- Architectural design feedback
- Performance optimization suggestions

This restriction ensures focused evaluation and reduces ambiguity in labeling.

8.3 Dataset

To prevent memorization effects and ensure fair evaluation, the dataset is divided into separate training (retrieval corpus) and evaluation repositories.

Data Sources Included:

1. Project Documentation
 - CONTRIBUTING.md
 - STYLE_GUIDE.md
 - Developer documentation
2. Relevant PEP8 Sections
 - Indentation rules
 - Naming conventions
 - Import organization
 - Mutable default argument guidelines
 - Docstring formatting standards
3. Static Analysis Rule Explanations
 - Pylint rule descriptions
 - Flake8 rule descriptions
 - Mapping of rule IDs to violation categories
4. Accepted Style-Related Review Comments
 - Extracted only from training repositories
 - Must correspond to one of the five violation categories

Chunking Strategy

- 200–400 tokens per chunk
- One rule/principle per chunk
- Minimal semantic overlap
- Metadata stored with each chunk:
 - Category
 - Source type
 - Repository origin

This ensures precise retrieval and controlled grounding.

8.4 Evaluation Metrics

The system is evaluated across multiple dimensions to ensure comprehensive analysis.

Issue Detection Accuracy

Evaluation is performed at the violation-instance level.

A prediction is considered correct if the generated comment identifies the same violation category as the corresponding human review comment.

Metrics reported:

- Precision (per category)
- Recall (per category)
- F1-score (per category)
- Macro-averaged F1-score

Grounding Rate

Grounding measures whether the generated comment explicitly references relevant retrieved knowledge.

A comment is considered grounded if it:

- References a retrieved guideline or rule
- Explicitly cites a project-specific standard
- Aligns with retrieved documentation content

Grounding Rate =

(Number of grounded comments / Total generated comments)

Hallucination Rate

A comment is classified as hallucinated if:

- It identifies a violation not present in the diff
- It references irrelevant or unsupported rules
- It introduces fabricated project policies

Hallucination Rate =

(Number of hallucinated comments / Total generated comments)

Semantic Alignment with Human Reviews

To measure alignment with reviewer intent:

- BERTScore F1 is computed between:
 - Generated comment
 - Corresponding human review comment

If the similarity exceeds a predefined threshold, the generated comment is considered semantically aligned.

Latency Analysis

To evaluate practical deployment feasibility:

- Retrieval latency
- Generation latency
- Total inference time

are recorded and compared between baseline LLM and RAG-based LLM.

9. Expected Outcomes

The proposed system is expected to demonstrate measurable improvements over both static analysis tools and a baseline LLM.

Improved Detection Accuracy

The RAG-based model is expected to achieve:

- Higher macro F1-score than the baseline LLM
- Improved recall for documentation and naming-related violations

Higher Grounding Rate

By incorporating project-specific retrieval, the RAG-based model should:

- Explicitly reference applicable style rules
- Demonstrate stronger contextual justification

Reduced Hallucination

Retrieval-based constraints are expected to:

- Reduce false-positive style suggestions

- Minimize unsupported or irrelevant guideline references

Project-Aware Comment Generation

Generated comments should:

- Reflect repository-specific conventions
- Match the tone and structure of human reviewers
- Provide constructive, localized feedback

Practical Prototype

The final deliverable will include:

- A functional RAG-based code review prototype
- A structured evaluation dataset
- Comparative experimental results
- Quantitative grounding and hallucination analysis
- Reproducible data collection pipeline